



Sistemas Embebidos I

Otoño 2026

Práctica 1: Inside the MCU

Camila Rodríguez Rosas

Ana Karen Rojas Ledesma

Jonathan Hernández Lazcano

sábado, 22 de agosto de 2026

Índice

Práctica 1: Inside the MCU	2
Introducción	2
Descripción de la práctica	2
Objetivo.....	2
Marco teórico	2
Desarrollo.....	7
Materiales	7
Esquemático	7
Desarrollo.....	7
Resultados	13
Actividad extra	14
Conclusiones.....	15
Declaración de IA	16
Referencias	16

Práctica 1: Inside the MCU

Introducción

Descripción de la práctica

La práctica consiste en la realización de un Blink directo y por toggle. Se enfoca en la comprensión y aplicación de SDK (Software Development Kit) y SIO (Single-cycle Input/Output); conceptos de arquitectura de software y hardware mediante el entorno de simulación de Wokwi. De esta forma, se pretende medir el ancho de pulso obtenido de cada simulación para analizar si existe alguna discrepancia entre los valores teóricos y los experimentales; así como el origen.

Objetivo

1. Escribir un blink (directo y por toggle) usando SDK.
2. Escribir un blink (directo y por toggle) usando acceso directo a registros.
3. Medir el ancho de pulso con el Logic Analyzer y Surfer Project.

Marco teórico

Microcontroladores vs. Microprocesadores

Los microprocesadores son circuitos integrados diseñados para ser Procesadores de Propósito General (GPP). Es decir, son capaces de manejar grandes cantidades de información de cualquier tipo, esto implica que son capaces de manejar bytes, palabras, números complejos, enteros, entre otros; para posteriormente usarlos en cualquier programa. Estos actúan como el cerebro de cualquier sistema computacional, siendo una “minicomputadora” ya que incorpora casi todas las funciones de un CPU.

Por otro lado, el microcontrolador se considera un Procesador con Conjunto de Instrucciones Específicos para Aplicaciones (ASIP). Esto significa que, serán procesadores diseñados únicamente para la realización de tareas específicas con la máxima eficiencia. Estos son pequeñas computadoras en un circuito integrado único que contienen un procesador, memoria y periféricos input/out programables. En general, los microprocesadores necesitaran de componentes externos como memoria, memoria RAM y periféricos que los microcontroladores ya poseen.

Arquitectura general interna de los microcontroladores

Es necesario para el correcto funcionamiento de un microcontrolador que estos cuenten con los siguientes recursos:

- CPU
- Unidad de memoria
- Puertos I/O
- Comunicación en serie
- Reloj
- Temporizador
- Detector de reinicio y baja tensión
- Convertidor analógico-digital

i. CPU

Un microcontrolador requiere de un microprocesador. El microprocesador será la computadora que utilizará el microcontrolador para correr los programas diseñado. Para ello, utiliza la memoria del programa y registros que permitirán la ejecución de la unidad aritmética/lógica (ALU).

ii. Unidad de memoria

Para almacenar los programas o información a ejecutar, los microcontroladores cuentan con un chip de memoria interna. Aunque en ciertos casos esta no llega a ser suficiente y es por ello, por lo que se agrega una unidad de memoria externa.

iii. Puertos I/O

Los microcontroladores poseen la capacidad de conectar el software con la realidad, esto se logra a través del bus de la memoria interna y su conexión con cientos de pines dentro del chip. Internamente a dichos pines se les llama puertos, mientras que la unidades externas y visibles serán llamados dispositivos Input/Output (I/O) o periféricos. De manera general, la conexión entre los programas y el mundo externo conlleva tres componentes: puertos, circuitos de interfaz, periféricos.

Los puertos son, de forma simplificada, buses (pistas) que se conectan únicamente a un solo dispositivo I/O. Estos, son registros de datos dentro del microcontrolador que pueden ser de entrada, de salida o bidireccionales; y que a su vez contienen su propia dirección asociada a un registro de datos permitiendo que cada pin se establezca con input u output individualmente.

Sin embargo, existe un gran reto en el tipo de datos que podemos obtener o establecer entre los pines y los puertos, que es la conversión analógica-digital. Aquí es donde intervienen los circuitos de interfaz permitiendo la conversión del voltaje o la corriente en datos binarios, o viceversa.

Finalmente, los periféricos o pine I/O serán físicamente los dispositivos proporcionados por el microcontrolador para conectar el software con el mundo real.

iv. Reloj

Los microcontroladores contienen un oscilador RC interno o un temporizador externo, como un cristal de cuarzo, que al momento de conectar la fuente de alimentación comienza a funcionar y es parte vital para la sincronización del temporizador del sistema y del CPU y sus ciclos. De esta forma, la frecuencia del oscilador determinará la velocidad con la que cada programa y acción sea ejecutada.

Arquitectura Raspberry Pi Pico 2

- Dual Cortex-M33 or Hazard3 processors at 150 MHz
- 520 kB on-chip SRAM, in 10 independent banks
- 8 kB of one-time-programmable storage (OTP)
- Up to 16 MB of external QSPI flash or PSRAM through dedicated QSPI bus
- Additional 16 MB flash or PSRAM through optional second chip-select
- On-chip switched-mode power supply to generate core voltage
- Optional low-quiescent-current LDO mode for sleep states
- 2 × on-chip PLLs for internal or external clock generation
- GPIOs are 5 V-tolerant (powered) and 3.3 V-failsafe (unpowered)

Periféricos:

- 2 × UARTs
- 2 × SPI controllers
- 2 × I2C controllers
- 24 × PWM channels
- USB 1.1 controller and PHY, with host and device support
- 12 × PIO state machines
- 1 × HSTX peripheral

GPIO (General Purpose Input/Output)

Un GPIO es un pin de propósito general que puede configurarse como entrada o salida digital bajo control de software. En modo salida, un pin push-pull usa transistores complementarios (PMOS/NMOS) para "empujar" la señal a alto (fuente de corriente) o "tirlarla" a bajo (sumidero); en modo entrada, lee el estado eléctrico externo.

a) Arquitectura del periférico GPIO

Cada GPIO puede ser controlado directamente por software (desde los procesadores) o por uno de varios bloques funcionales. La función asignada a cada GPIO se selecciona mediante `gpio_set_function`. Cada pin tiene un multiplexor de funciones (FUNCSEL). El sistema GPIO consta de tres grupos de registros de hardware:

pads_bank0_hw (PADS_BANK0): controla características eléctricas (resistencias de pull, drive strength, slew rate, input enable, histéresis Schmitt, output disable).

io_bank0_hw (IO_BANK0): controla el multiplexado de funciones (muxing) y los overrides de señal; escribe el valor de función en el campo FUNCSEL de `io_bank0_hw->io[gpio].ctrl`.

sio_hw (SIO): provee acceso de ciclo único a los valores y dirección de los GPIO para I/O controlado por software.

Software Development Kit (SDK)

El SDK es el conjunto de archivos, librerías y sistemas de compilación necesarios que proporciona Raspberry Pi para la escritura de códigos destinados a microcontroladores. Está diseñado para ofrecer una API y un entorno de programación familiar tanto para desarrolladores C embebidos como no embebidos. Un solo programa corre en el dispositivo a la vez y comienza con un método `main()` convencional.

a) Arquitectura y organización

El SDK está organizado jerárquicamente. Las librerías se agrupan en categorías:

- **Librerías de soporte de hardware** (`hardware_XXX`): proporcionan las APIs para interactuar con cada periférico físico.
- **Librerías de alto nivel / de agregación** (`pico_XXX`): son librerías INTERFACE que agregan un conjunto de funcionalidad en bloques consumibles. La más común es `pico_stdlib`, que agrega un subconjunto central de librerías usadas por la mayoría de los ejecutables (incluye `hardware_gpio`, `pico/time.h`, etc.). Otras son `pico_multicore`, `pico_time`.
- **Librerías runtime**: rutinas de bajo nivel que empaquetan funcionalidad común a la mayoría de aplicaciones.

b) Funciones clave de GPIO en el SDK

`gpio_init(gpio)`: inicializa un GPIO para I/O de software (habilita I/O y fija la función a `GPIO_FUNC_SIO`, limpia output enable y valor de salida).

`gpio_set_dir(gpio, out)`: fija la dirección (entrada/salida).

`gpio_put(gpio, value)`: escribe un valor en un pin (usa `gpio_set_mask/gpio_clr_mask`).

`gpio_set_function(gpio, fn)`: selecciona la función del multiplexor (FUNCSEL).

`gpio_xor_mask(mask)`: hace toggle atómico de los pines de la máscara vía el alias `TOGL`.

`gpio_set_drive_strength`, `gpio_set_slew_rate`, `gpio_pull_up`, `gpio_pull_down`.

SIO (Single-cycle I/O)

El bloque Single-cycle IO (SIO) contiene varios periféricos que requieren acceso de baja latencia y determinístico desde los procesadores. En el RP2040 se accede vía el IOPORT de cada procesador: un puerto de bus auxiliar del Cortex-M0+ que puede realizar lecturas y escrituras rápidas de 32 bits. Cada procesador accede a su IOPORT con instrucciones normales de load y store, dirigidas al segmento de direcciones especial IOPORT, `0xd0000000...0xdfffffff`. A diferencia de los periféricos en el bus APB, las operaciones sobre registros SIO se completan típicamente en un solo ciclo de reloj. Por eso el SIO es el camino más rápido para leer y escribir GPIOs. El SIO no soporta el acceso atómico normal por alias del bus, pero algunos registros individuales (como GPIO) tienen alias set, clear y XOR propios.

a) Mapa de registros

Los registros del SIO comienzan en la dirección base `0xd0000000` (definida como `SIO_BASE` en el SDK). En el RP2040 (offsets desde `SIO_BASE`):

- `CPUID (0x000)`: identificador del núcleo procesador.
- `GPIO_IN (0x004)`: valor de entrada de los GPIO.
- `GPIO_HI_IN (0x008)`: valor de entrada de los pines QSPI.
- `GPIO_OUT (0x010)`: valor de salida de los GPIO.
- `GPIO_OUT_SET (0x014)`: pone a 1 (set atómico) bits de `GPIO_OUT`.
- `GPIO_OUT_CLR (0x018)`: pone a 0 (clear atómico) bits de `GPIO_OUT`.
- `GPIO_OUT_XOR (0x01c)`: XOR atómico sobre `GPIO_OUT` (alias `TOGL`).
- `GPIO_OE (0x020)`: output enable.
- `GPIO_OE_SET (0x024)`, `GPIO_OE_CLR (0x028)`, `GPIO_OE_XOR (0x02c)`: set/clear/XOR atómicos del output enable.

Los registros OUT y OE también tienen alias atómicos SET, CLR y XOR, lo que permite actualizar un subconjunto de pines en una sola operación; esto es vital para el acceso paralelo seguro entre los dos núcleos.

b) Cómo funcionan los punteros

sio_hw-> en el código. El SDK define una estructura C que mapea los registros del SIO y un puntero `sio_hw` a la base. Así, `sio_hw->gpio_set` corresponde a la dirección `0xd0000014`, `sio_hw->gpio_clr` a `0xd0000018` y `sio_hw->gpio_togl` a `0xd000001c`. Las funciones inline del SDK usan estos punteros:

```
gpio_xor_mask(mask) → sio_hw->gpio_togl = mask;
gpio_put_masked(mask, value) → sio_hw->gpio_togl = (sio_hw->gpio_out ^ value) & mask;
gpio_put_all(value) → sio_hw->gpio_out = value;
gpio_set_dir_out_masked(mask) → sio_hw->gpio_oe_set = mask;
```

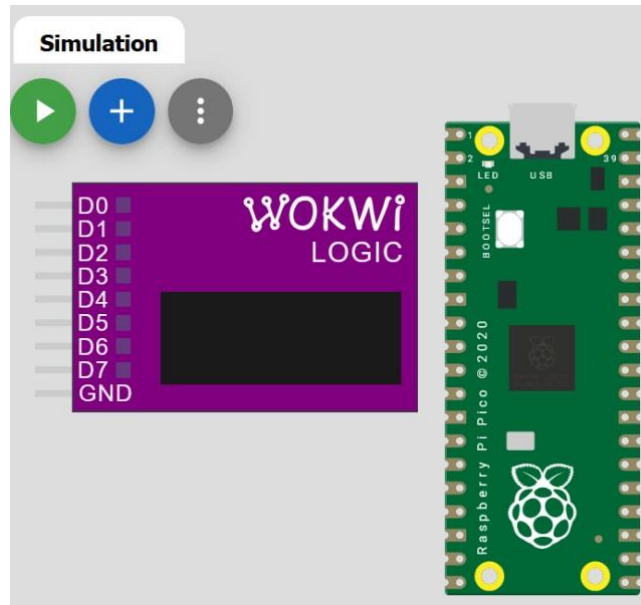
Desarrollo

Materiales

Entornos de simulación y software:

- Wokwi Simulation Platform
- Surfer Project

Esquemático



Desarrollo

Parte 1: Blink directo usando SDK

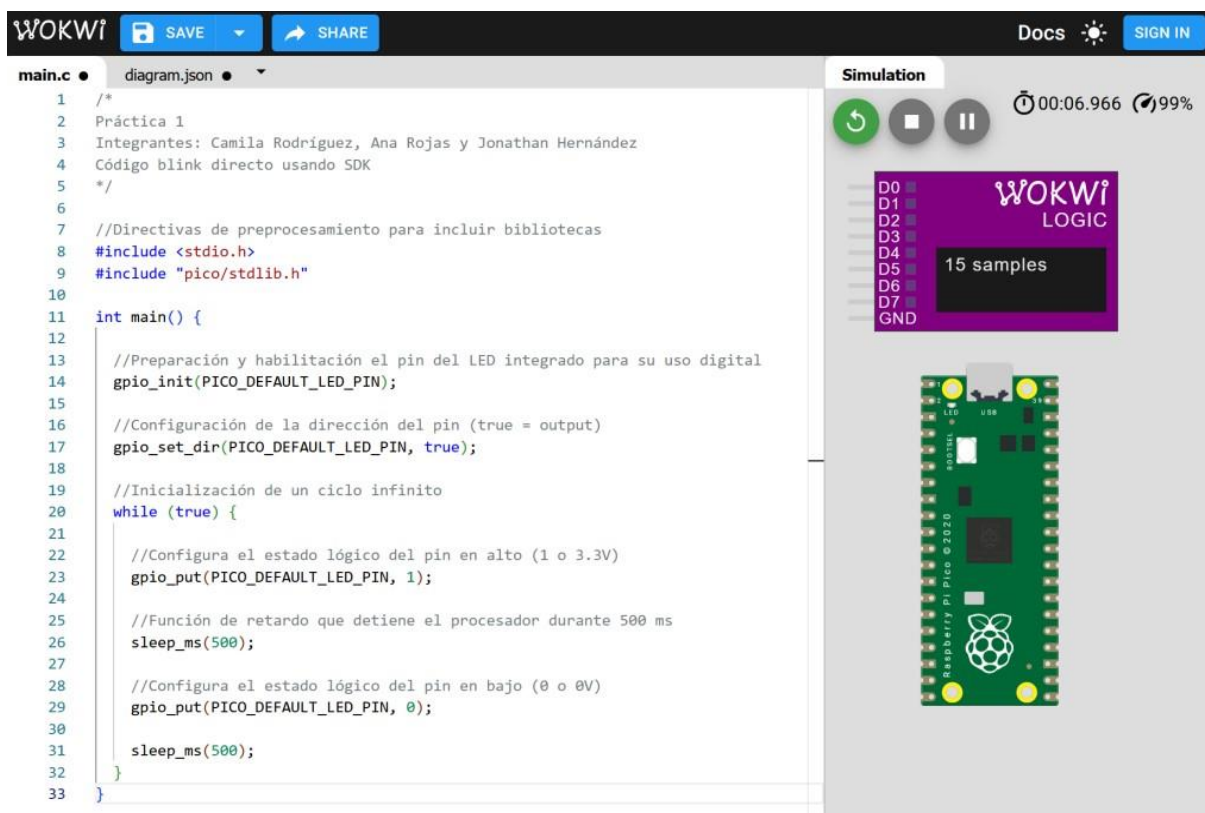
i. Código escrito

```
/*
Práctica 1
Integrantes: Camila Rodríguez, Ana Rojas y Jonathan Hernández
Código blink directo usando SDK
*/
//Directivas de preprocesamiento para incluir bibliotecas
#include <stdio.h>
#include "pico/stdlib.h"
int main() {
    //Preparación y habilitación el pin del LED integrado para su uso
    digital
    gpio_init(PICO_DEFAULT_LED_PIN);
    //Configuración de la dirección del pin (true = output)
    gpio_set_dir(PICO_DEFAULT_LED_PIN, true);
```



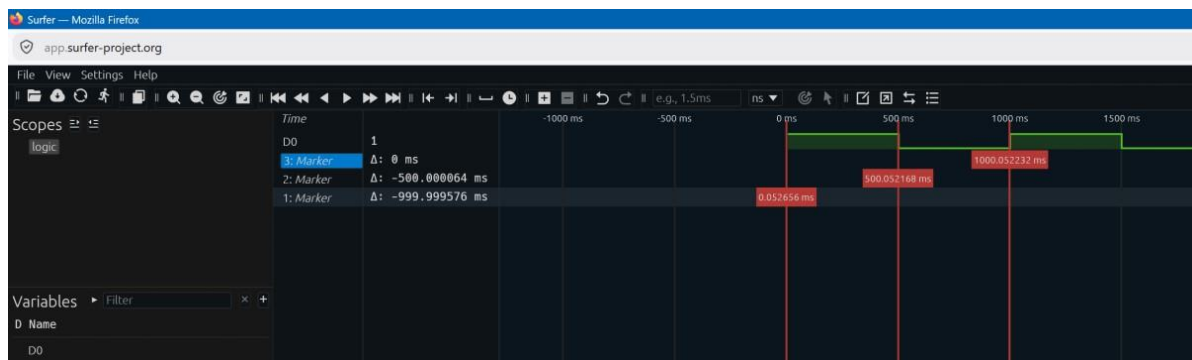
```
//Iniciación de un ciclo infinito
while (true) {
    //Configura el estado lógico del pin en alto (1 o 3.3V)
    gpio_put(PICO_DEFAULT_LED_PIN, 1);
    //Función de retardo que detiene el procesador durante 500 ms
    sleep_ms(500);
    //Configura el estado lógico del pin en bajo (0 o 0V)
    gpio_put(PICO_DEFAULT_LED_PIN, 0);
    sleep_ms(500);
}
}
```

ii. Simulación en Wokwi



The screenshot shows the Wokwi web interface for simulating a Raspberry Pi Pico. On the left, the 'main.c' file is open, displaying C code for a blink application. The code includes headers for stdio.h and pico/stdlib.h, and uses gpio_init, gpio_set_dir, gpio_put, and sleep_ms functions to toggle the onboard LED. On the right, the 'Simulation' window shows a 3D model of the Raspberry Pi Pico board. Above the board, a 'WOKWI LOGIC' window displays a digital signal trace for pin D0, showing a square wave with a period of 1000ms and a pulse width of 500ms. The simulation is running at 99% speed.

iii. Medición en Surfer Project

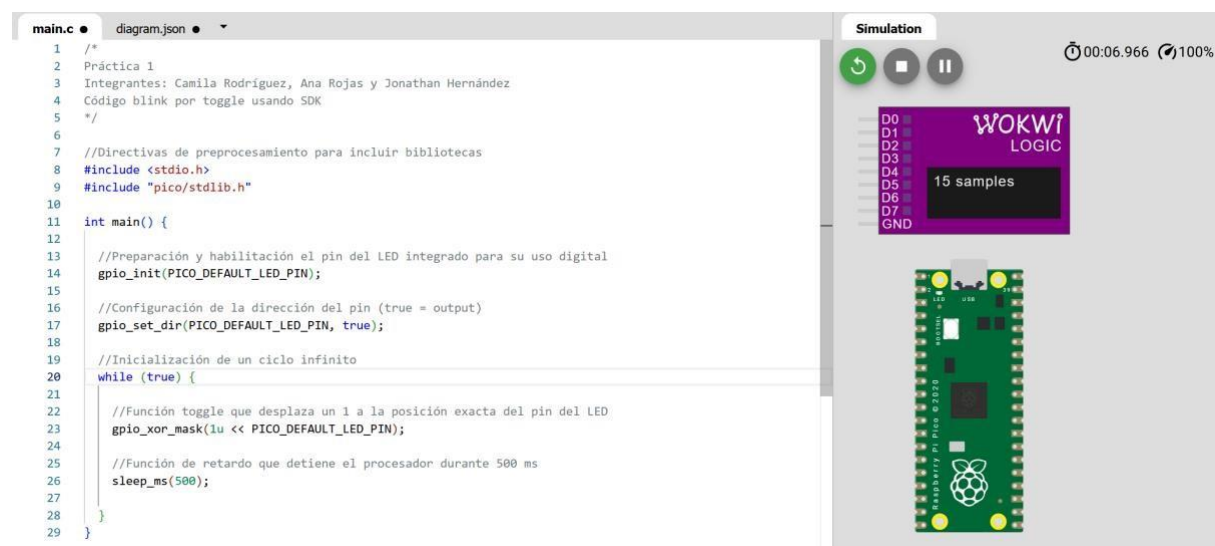


Parte 2 : Blink por toggle usando SDK

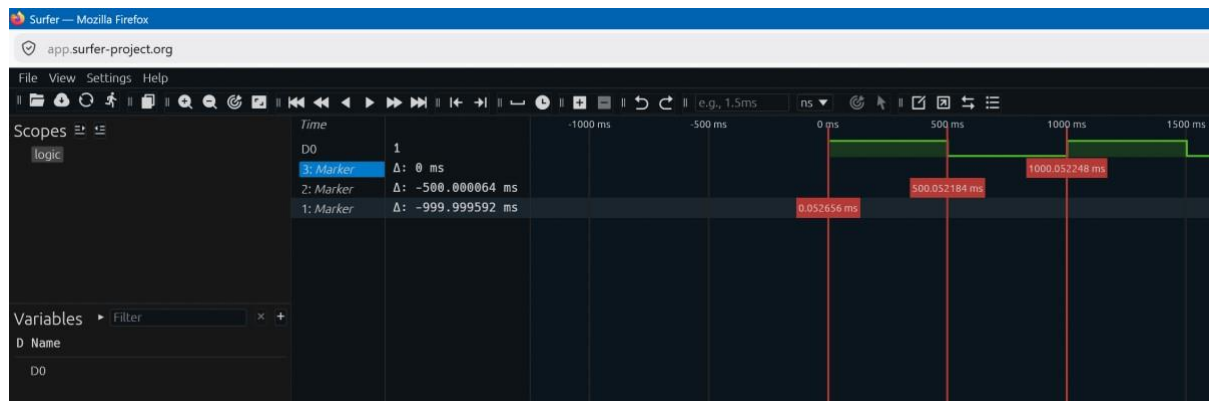
i. Código escrito

```
/*
Práctica 1
Integrantes: Camila Rodríguez, Ana Rojas y Jonathan Hernández
Código blink por toggle usando SDK
*/
//Directivas de preprocesamiento para incluir bibliotecas
#include <stdio.h>
#include "pico/stdlib.h"
int main() {
    //Preparación y habilitación el pin del LED integrado para su uso
    digital
    gpio_init(PICO_DEFAULT_LED_PIN);
    //Configuración de la dirección del pin (true = output)
    gpio_set_dir(PICO_DEFAULT_LED_PIN, true);
    //Inicialización de un ciclo infinito
    while (true) {
        //Función toggle que desplaza un 1 a la posición exacta del pin del LED
        gpio_xor_mask(1u << PICO_DEFAULT_LED_PIN);
        //Función de retardo que detiene el procesador durante 500 ms
        sleep_ms(500);
    }
}
```

ii. Simulación en Wokwi



iii. Medición en Surfer Project



Parte 3: Blink directo usando acceso directo a registros

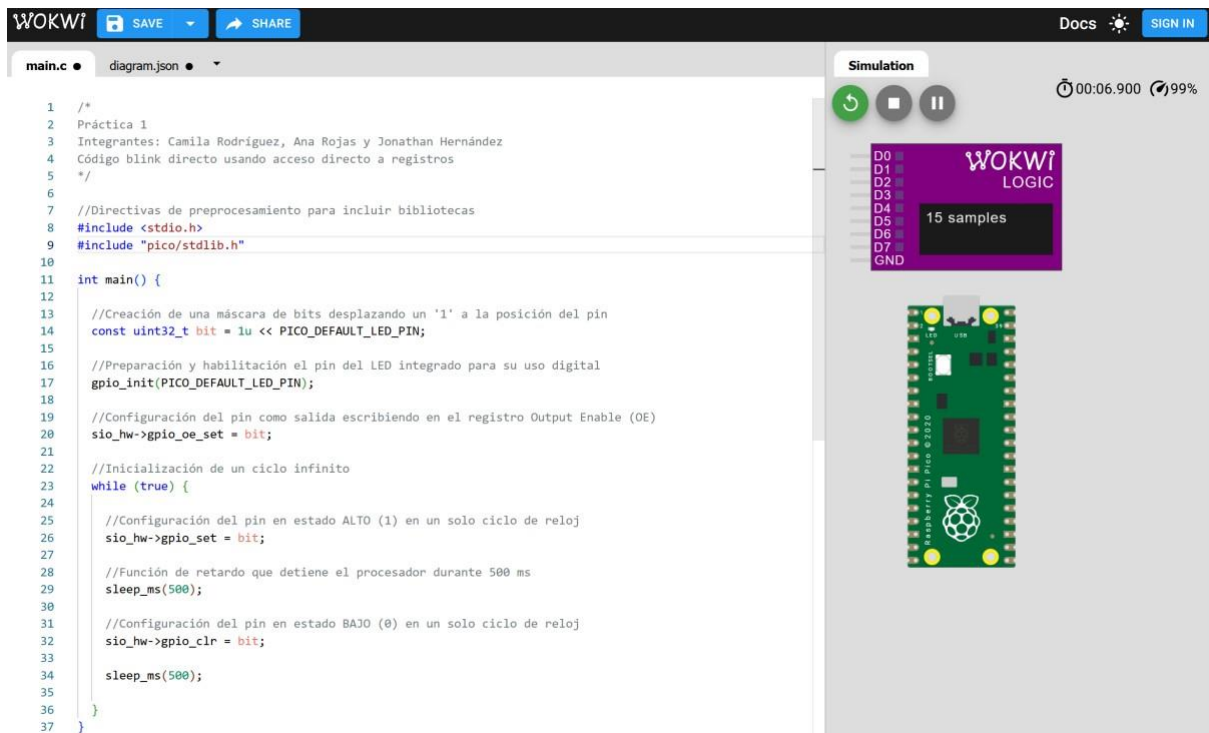
i. Código escrito

```

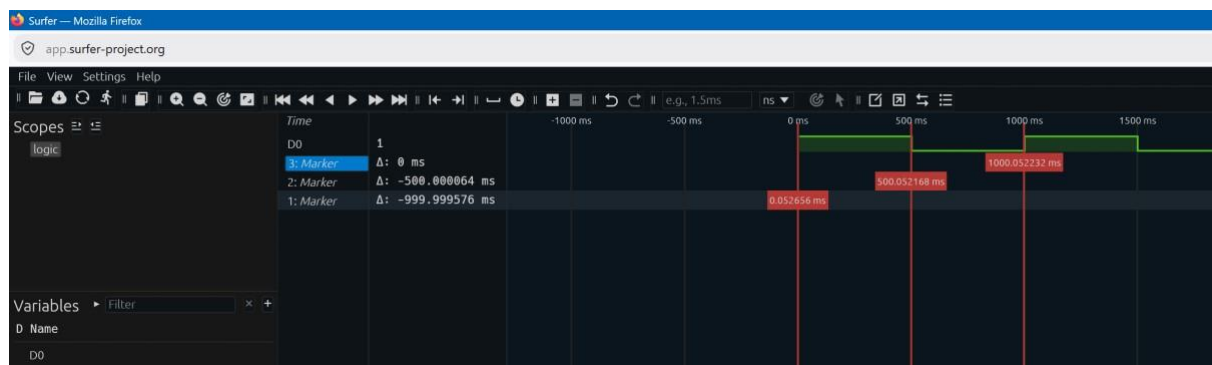
/*
Práctica 1
Integrantes: Camila Rodríguez, Ana Rojas y Jonathan Hernández
Código blink directo usando acceso directo a registros
*/
//Directivas de preprocesamiento para incluir bibliotecas
#include <stdio.h>
#include "pico/stdlib.h"
int main() {
    //Creación de una máscara de bits desplazando un '1' a la posición del
    pin
    const uint32_t bit = 1u << PICO_DEFAULT_LED_PIN;
    //Preparación y habilitación el pin del LED integrado para su uso
    digital
    gpio_init(PICO_DEFAULT_LED_PIN);
    //Configuración del pin como salida escribiendo en el registro Output
    Enable (OE)
    sio_hw->gpio_oe_set = bit;
    //Inicialización de un ciclo infinito
    while (true) {
        //Configuración del pin en estado ALTO (1) en un solo ciclo de
        reloj
        sio_hw->gpio_set = bit;
        //Función de retardo que detiene el procesador durante 500 ms
        sleep_ms(500);
        //Configuración del pin en estado BAJO (0) en un solo ciclo de
        reloj
        sio_hw->gpio_clr = bit;
        sleep_ms(500);
    }
}

```

ii. Simulación en Wokwi



iii. Medición en Surfer Project



Parte 4: Blink por toggle usando acceso directo a registros

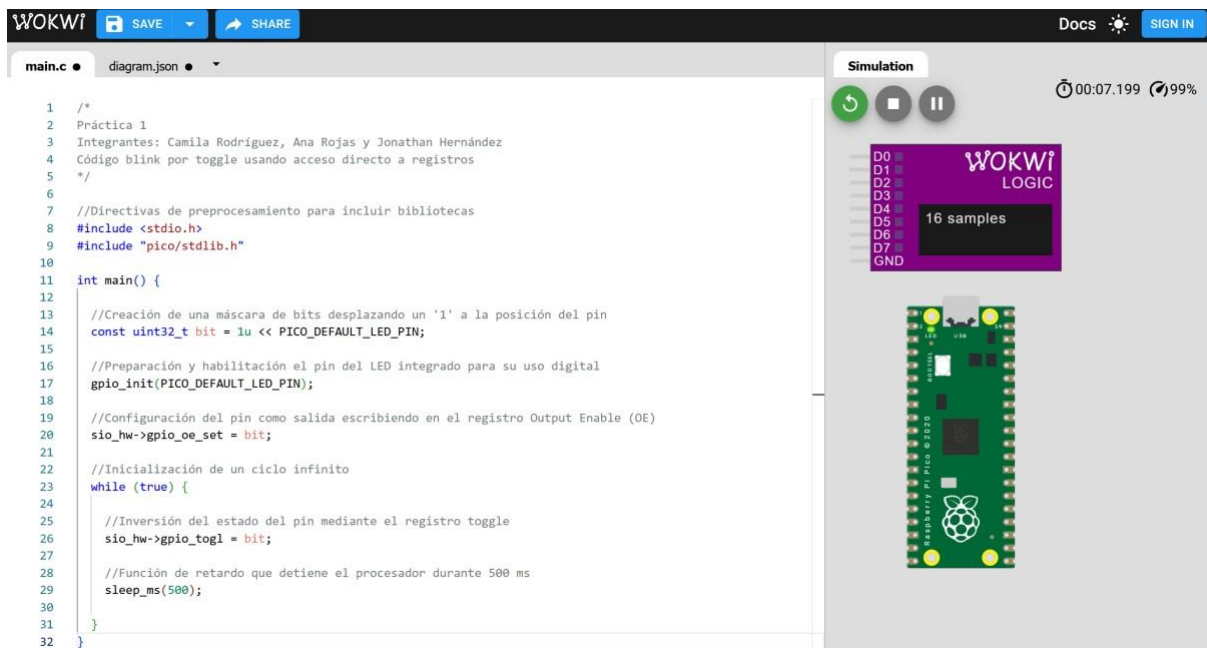
i. Código escrito

```

/*
Práctica 1
Integrantes: Camila Rodríguez, Ana Rojas y Jonathan Hernández
Código blink por toggle usando acceso directo a registros
*/
//Directivas de preprocesamiento para incluir bibliotecas
#include <stdio.h>
#include "pico/stdlib.h"
int main() {
  
```

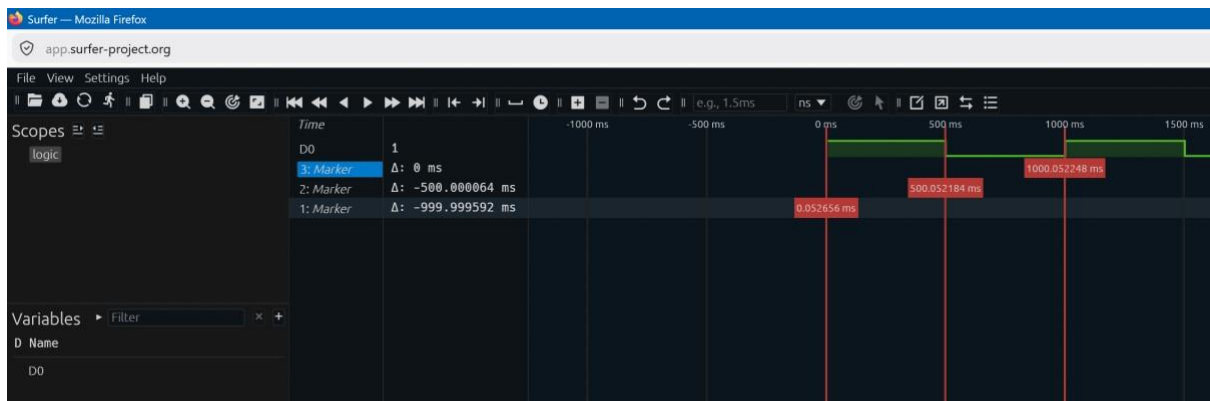
```
//Creación de una máscara de bits desplazando un '1' a la posición del
pin
const uint32_t bit = 1u << PICO_DEFAULT_LED_PIN;
//Preparación y habilitación el pin del LED integrado para su uso
digital
gpio_init(PICO_DEFAULT_LED_PIN);
//Configuración del pin como salida escribiendo en el registro Output
Enable (OE)
sio_hw->gpio_oe_set = bit;
//Inicialización de un ciclo infinito
while (true) {
    //Inversión del estado del pin mediante el registro toggle
    sio_hw->gpio_togl = bit;
    //Función de retardo que detiene el procesador durante 500 ms
    sleep_ms(500);
}
}
```

ii. Simulación en Wokwi



The screenshot displays the Wokwi web-based simulation environment. On the left, the 'main.c' file is open, showing a C program for a Raspberry Pi Pico. The code includes headers for `<stdio.h>` and `"pico/stdlib.h"`. The `main` function initializes the GPIO pin, sets the output enable register, and enters an infinite loop where it toggles the pin state and sleeps for 500 ms. On the right, the 'Simulation' panel shows a Raspberry Pi Pico board. Above the board, the 'WOKWI LOGIC' window displays 16 samples of the digital signal. The simulation is running, as indicated by the play button and the timer showing 00:07.199.

iii. Medición en Surfer Project



Resultados

Parte 1: Blink directo usando SDK

Parámetro	Valor esperado(del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	499.999512 ms
Tiempo en bajo	500 ms	500.000064 ms
Periodo	1000 ms	999.999576 ms
Frecuencia	1 Hz	1.00000424 Hz

Parte 2: Blink por toggle usando SDK

Parámetro	Valor esperado(del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	499.999528 ms
Tiempo en bajo	500 ms	500.000064 ms
Periodo	1000 ms	999.999592 ms
Frecuencia	1 Hz	1.000000408 Hz

Parte 3: Blink directo usando acceso directo a registros

Parámetro	Valor esperado(del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	499.999512 ms
Tiempo en bajo	500 ms	500.000064 ms
Periodo	1000 ms	999.999576 ms
Frecuencia	1 Hz	1.00000424 Hz

Parte 4: Blink por toggle usando acceso directo a registros

Parámetro	Valor esperado(del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	499.999528 ms
Tiempo en bajo	500 ms	500.000064 ms
Periodo	1000 ms	999.999592 ms
Frecuencia	1 Hz	1.000000408 Hz

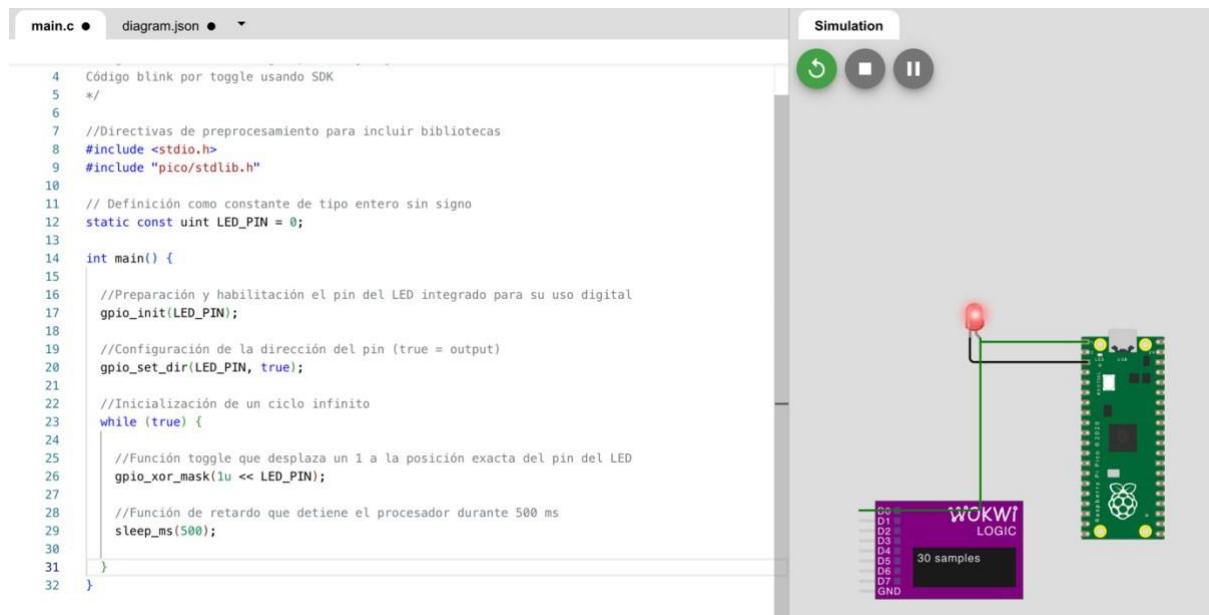
Actividad extra

Hacer un blink conectando un LED externo a cualquier PIN de GPIO

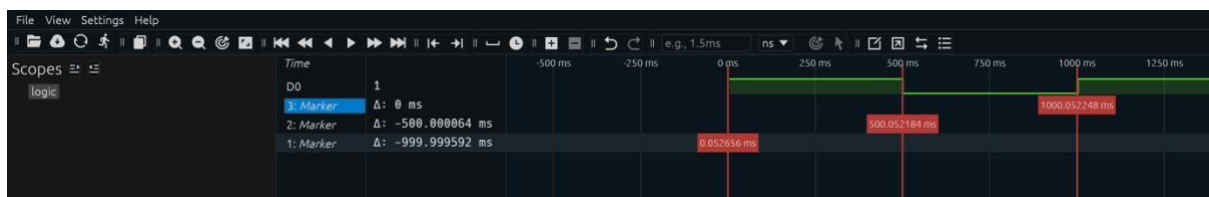
i. Código escrito

```
/*
Práctica 1
Integrantes: Camila Rodríguez, Ana Rojas y Jonathan Hernández
Ejercicio Extra
*/
//Directivas de preprocesamiento para incluir bibliotecas
#include <stdio.h>
#include "pico/stdlib.h"
// Definición como constante de tipo entero sin signo
static const uint LED_PIN = 0;
int main() {
    //Preparación y habilitación el pin del LED integrado para su uso
    digital
    gpio_init(LED_PIN);
    //Configuración de la dirección del pin (true = output)
    gpio_set_dir(LED_PIN, true);
    //Inicialización de un ciclo infinito
    while (true) {
        //Función toggle que desplaza un 1 a la posición exacta del pin
        del LED
        gpio_xor_mask(1u << LED_PIN);
        //Función de retardo que detiene el procesador durante 500 ms
        sleep_ms(500);
    }
}
```


ii. Simulación en Wokwi



iii. Medición en Surfer Project



iv. Tabla comparativa

Parámetro	Valor esperado(del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	499.999528 ms
Tiempo en bajo	500 ms	500.000064 ms
Periodo	1000 ms	999.999592 ms
Frecuencia	1 Hz	1.000000408 Hz

Conclusiones

En conclusión, los microcontroladores son una herramienta tecnológica ampliamente utilizada para la realización de tareas específicas que involucran una interacción entre programas de software y el mundo real. Al utilizar el entorno de simulación de una Raspberry Pi Pico mediante Wokwi nos fue posible cumplir exitosamente con la programación de un Blink directo y por toggle mediante SDK y SIO. Dicho resultado podemos corroborarlo en las figuras presentadas en la metodología, permitiéndonos afirmar que Wokwi es una funcional para la simulación y el autoaprendizaje del uso de microcontroladores como la Raspberry Pi Pico.

Al enfocarnos en la escritura de los códigos, cada uno de ellos pretendía generar un blink con los mismos valores. Es decir, independientemente de utilizar SDK o SIO se pretendía obtener de manera simulada una señal digital con valores de 500 ms en Alto y 500ms en Bajo, para así obtener una frecuencia de 1 Hz y un periodo de 1 s. Sin embargo, los valores medidos mediante Surfer Project indican discrepancia.

Al examinar el apartado de resultados observamos que todas las tablas presentan una discrepancia. Sin embargo, se observa que los códigos que utilizan la herramienta toggle obtienen un menor retaso y son idénticos, mientras que los códigos del blink directo conllevan un mayor retraso y también son idénticos. Examinando aun nuevamente los códigos escritos se observa que las versiones usando toggle contienen menos líneas que su contraparte haciendo un blink directo. Con ello, concluimos que la variación en los resultados se debe a la cantidad de líneas que se deban ejecutar provocando una sobrecarga de código.

Si bien, la herramienta toggle acorta las líneas de código que deben ser procesadas y ejecutadas, también se esperaría que en una práctica real sea aún más eficiente utilizar SIO. Ya que este permite una comunicación más rápida entre los pines I/O y el microprocesador, evitando ejecuciones “invisible” que conlleva el SDK. Por ende, SDK al ser diseñado por Raspberry Pi es un sistema muy amistoso para familiarizarse con el funcionamiento de los microcontroladores. Sin embargo, sería una mejor práctica cuidar la sobrecarga de procesamiento de código utilizando las rutas directas de conexión mediante SIO.

Declaración de IA

Herramientas de IA utilizadas y su uso:

1. Google Gemini:
 - a. Generación de propuestas del orden y temas expuestos dentro del marco teórico.
 - b. Generación de propuestas para los comentarios de cada código.
 - c. Generación y orden de referencias en formato apa
2. Claude:
 - a. Búsqueda de fuentes para el marco teórico

Referencias

Dawoud, S. (2010). Digital system design: Use of microcontroller. River Publishers.
<https://ebookcentral.proquest.com/lib/iberopueblaebooks/detail.action?docID=34001>

Kothari, D. P., Khan, G. K., & Rai, S. (2011). Embedded systems. New Age International Ltd.

<https://ebookcentral.proquest.com/lib/iberopueblaebbooks/detail.action?docID=3017432>

Ramírez Cruz, M. N. (s. f.). SE1 M1.1: Inside the MCU [Diapositivas de presentación]. Moodle, Universidad Iberoamericana Puebla.

https://aulasvirtuales.iberopuebla.mx/pluginfile.php/3525062/mod_resource/content/3/SE1_M1.1_Inside%20the%20MCU.pdf

Raspberry Pi Ltd. (s. f.). Raspberry Pi Pico-series documentation. Raspberry Pi Documentation. Recuperado el 22 de agosto de 2026, de <https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>

Raspberry Pi Ltd. (2024). RP2350 datasheet: A microcontroller by Raspberry Pi (Hoja de datos n.º RP-008373-DS-2). <https://pipassets.raspberrypi.com/categories/1214-rp2350/documents/RP-008373-DS-2rp2350-datasheet.pdf>

Raspberry Pi Ltd. (s.f.). The C/C++ SDK — Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/microcontrollers/c_sdk.html

Raspberry Pi Ltd. (s.f.). Raspberry Pi Pico SDK (Doxygen API docs). <https://www.raspberrypi.com/documentation/pico-sdk/>

Raspberry Pi Ltd. (2024). RP2350 datasheet: A microcontroller by Raspberry Pi. <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf> (Capítulo 9 GPIO)

Raspberry Pi Ltd. (2024). RP2040 datasheet: A microcontroller by Raspberry Pi (Sección 2.3.1 SIO; 2.3.1.7 List of Registers). <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>